Guía Definitiva de Arquitectura Limpia en React y TypeScript: Patrones, Estrategias y Mejores Prácticas para 2025

1. Filosofía Arquitectónica y Resumen Ejecutivo

El desarrollo de software frontend ha evolucionado desde la simple manipulación del DOM hacia la construcción de sistemas distribuidos complejos que residen en el navegador. React, con su naturaleza declarativa, y TypeScript, con su sistema de tipos estructural, forman la columna vertebral de la mayoría de las aplicaciones empresariales modernas. Sin embargo, la flexibilidad de React y la rigurosidad de TypeScript a menudo generan puntos de fricción donde los equipos divergen, creando bases de código fragmentadas, difíciles de mantener y propensas a errores.

Esta guía técnica, redactada desde la perspectiva de una mentoría avanzada en "Clean Code", tiene como objetivo codificar un conjunto de prácticas exhaustivas para 2025. El enfoque no es simplemente "hacer que funcione", sino construir una arquitectura que soporte la escalabilidad, la legibilidad y la refactorización segura a largo plazo. Analizaremos en profundidad la dicotomía entre interface y type, la gestión avanzada de eventos, la arquitectura de capas para servicios API, y los patrones de diseño como Componentes Compuestos (Compound Components) y Hooks Personalizados que permiten una separación de responsabilidades cristalina.

La premisa fundamental de este reporte es que el código limpio en React no es una preferencia estética, sino una necesidad económica. La deuda técnica acumulada por un tipado laxo (uso de any), componentes monolíticos y una gestión de estado descentralizada se traduce directamente en una menor velocidad de entrega y una mayor fragilidad del sistema a medida que este crece. A través de este documento, transformaremos conceptos abstractos en directrices tangibles, respaldadas por un análisis profundo de las tendencias actuales y la investigación técnica.[1]

2. El Fundamento de la Seguridad de Tipos en el Ecosistema React

TypeScript no es simplemente una herramienta de autocompletado; es un contrato vinculante que asegura la integridad de los datos a través de toda la aplicación. En 2025, la práctica estándar dicta que la seguridad de tipos debe ser prioritaria, evitando compromisos que introduzcan inestabilidad en tiempo de ejecución. El objetivo es aprovechar la inferencia de tipos para reducir la verbosidad sin sacrificar la seguridad, permitiendo que el compilador actúe como una primera línea de defensa contra errores lógicos.[1]

2.1 La Dialéctica Fundamental: Interface versus Type

Uno de los debates más persistentes y matizados en la comunidad de TypeScript es la elección entre interface y type (alias de tipo). Aunque superficialmente parecen intercambiables, sus diferencias estructurales y de comportamiento bajo el compilador tienen implicaciones profundas para la arquitectura de la aplicación.

2.1.1 Interfaces: Extensibilidad y Modelado Orientado a Objetos

La palabra clave interface tiene sus raíces en la programación orientada a objetos. Su característica definitoria es la capacidad de ser extendida y fusionada. La "fusión de declaraciones" (declaration merging) permite que múltiples declaraciones de la misma interfaz se combinen automáticamente en una sola definición.

Esta característica hace que las interfaces sean la elección superior y casi obligatoria para definir APIs públicas de librerías. Si usted está autorizando una biblioteca de componentes (por ejemplo, un sistema de diseño interno), usar interfaces permite a los equipos consumidores aumentar las definiciones existentes (como agregar una propiedad personalizada al tema global) sin necesidad de bifurcar el código fuente. Además, las interfaces suelen ofrecer un mejor rendimiento en la comprobación de errores del compilador de TypeScript, ya que almacenan en caché la forma del objeto, a diferencia de los tipos que

se evalúan recursivamente.[3]

2.1.2 Alias de Tipo (Type): Composición Funcional y Uniones

Por el contrario, los type aliases son más flexibles y poderosos para describir formas de datos en un paradigma funcional, que es el que domina el desarrollo moderno de React. El consenso actual para el desarrollo de aplicaciones generales —específicamente para props internos de componentes y definición de estados— es utilizar type.

La razón principal es la capacidad de los alias de tipo para definir **Tipos de Unión** (Union Types) y **Tipos de Intersección**. Una unión (type Status = 'success' | 'error') es fundamental para modelar estados finitos, algo que las interfaces no pueden hacer directamente. Además, los tipos permiten definir tuplas y alias de primitivos, lo cual es indispensable, por ejemplo, al tipar el valor de retorno de un Custom Hook que devuelve un array [valor, setter].[5]

Tabla 1: Análisis Comparativo Estratégico - Interface vs. Type

Característica	Interface	Type Alias	Contexto Recomendado de Uso
Extensibilidad	Soporta Fusión de Declaraciones (abierto)	Cerrado (no puede reabrirse)	Usar **Interface** para APIs públicas de librerías.
Composición	Palabra clave extends	Intersección (&)	Usar **Type** para componer props complejos en apps.
Uniones	No puede definir uniones directamente	Uso nativo (`	`)
Primitivos	No puede aliar primitivos	Puede aliar primitivos	Usar **Type** para tipos utilitarios.
Mapeo	No soporta Mapped Types	Soporta Mapped Types (in keyof)	Usar **Type** para transformaciones

	directos		de objetos.
Mensajes de Error	Generalmente más concisos	Pueden ser verbosos	**Interface** preferida para legibilidad en errores simples.

2.1.3 Recomendación del Mentor

La regla general para 2025 es: **"Usa interface hasta que necesites type"**. Sin embargo, en el contexto específico de una aplicación React (no una librería), la balanza se inclina hacia type debido a la prevalencia de uniones y la necesidad de evitar la fusión accidental de declaraciones, que puede ser un comportamiento implícito peligroso en grandes bases de código.[3]

2.2 Tipado de Hijos (Children): El Estándar ReactNode

El tipado correcto de la prop children es una fuente frecuente de confusión. Históricamente, los desarrolladores han oscilado entre JSX.Element, ReactElement y ReactNode. La distinción es crucial para la flexibilidad del componente.

La investigación indica inequívocamente que ReactNode es el tipo más robusto y permisivo para children. Representa "cualquier cosa que React puede renderizar": elementos, cadenas de texto, números, fragmentos, portales, null, undefined y arrays de estos tipos.[7] Restringir los hijos a JSX.Element (que es estrictamente el valor de retorno de React.createElement) causaría errores de tipo innecesarios si un consumidor intentara pasar un simple string ("Guardar") o un número como hijo de un botón.[3]

2.2.1 El Ocaso de React.FC y PropsWithChildren

En versiones anteriores, el tipo React.FC (Functional Component) era ubicuo. Este tipo añadía implícitamente la prop children a cualquier componente. Sin embargo, las mejores prácticas modernas desaconsejan explícitamente el uso de React.FC. La razón es que otorga una

definición de children incluso a componentes que no deberían aceptar hijos (componentes "hoja"), lo que debilita el contrato de la API del componente.[9]

De manera similar, aunque PropsWithChildren<P> es conveniente, a menudo es preferible definir children explícitamente en el tipo de props. Esto permite controlar si los hijos son obligatorios u opcionales, una distinción que PropsWithChildren (que los hace opcionales por defecto) oscurece. La claridad explícita (children: ReactNode) supera a la conveniencia implícita en bases de código mantenibles.[10]

Ejemplo de Patrón Recomendado:

TypeScript

```
type CardProps = {
  title: string;
  // Definición explícita: Este componente DEBE recibir hijos
  children: React.ReactNode;
};

// Sin React.FC, con inferencia de retorno
export const Card = ({ title, children }: CardProps) => {
  return (
    <div className="card">
      <h1>{title}</h1>
      {children}
    </div>
  );
};
```

---

3. Manejo Estricto de Eventos y Formularios

El manejo de eventos en formularios es un área donde el tipado "laxo" (uso de any o tipos genéricos incorrectos) introduce riesgos significativos. React utiliza SyntheticEvent, un wrapper sobre los eventos nativos del navegador para garantizar consistencia.

3.1 Especificidad en los Tipos de Eventos

Usar simplemente React.FormEvent o React.ChangeEvent es insuficiente. Se debe especificar el genérico del elemento HTML que dispara el evento, por ejemplo: React.ChangeEvent<HTMLInputElement>. Esto es vital porque informa a TypeScript sobre las propiedades disponibles en event.target (como value, checked, files), eliminando la necesidad de casting manual o suposiciones peligrosas.[12]

3.2 La Trampa de target vs. currentTarget

Un error arquitectónico común es confundir target con currentTarget.

- **target**: Es el elemento que originó el evento. Debido al burbujeo de eventos (bubbling), esto podría ser un dentro de un <button>, y no el botón mismo.
- **currentTarget**: Es el elemento al cual se adjuntó el manejador del evento (listener). En React, este siempre coincidirá con el elemento JSX donde se definió la prop onChange u onClick.

Para un tipado fiable y un comportamiento predecible, especialmente en elementos compuestos, se debe priorizar el uso de currentTarget cuando se accede a atributos del elemento manejador, ya que TypeScript puede garantizar su tipo basándose en el genérico proporcionado.[14]

Código Limpio en Manejadores de Eventos:

TypeScript

```
import React, { useState } from 'react';

export const SearchBar = () => {
  const = useState("");

  // Tipado preciso en la firma de la función
  const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {
    // Acceso seguro a currentTarget.value (garantizado string)
    setTerm(event.currentTarget.value);
```

```
  };

  return <input type="text" value={term} onChange={handleChange} />;
};
```

Este enfoque evita la inferencia implícita de any y documenta claramente qué tipo de elemento se espera que maneje esta función, facilitando la refactorización si el input cambiara a un textarea o un select en el futuro.[16]

---

4. Arquitectura de Componentes y Patrones de Diseño Avanzados

A medida que las aplicaciones escalan, la complejidad de los componentes aumenta exponencialmente. Sin patrones arquitectónicos estrictos, los componentes tienden a convertirse en "Objetos Dios" (God Objects): monolitos que manejan la obtención de datos, lógica de negocio compleja y renderizado de UI simultáneamente. La solución reside en la estricta separación de responsabilidades.

4.1 El Patrón de Componentes Compuestos (Compound Components)

El patrón de Componentes Compuestos se identifica como la solución principal para evitar la "Sopa de Props" (Prop Soup), donde un componente padre recibe docenas de props de configuración que simplemente pasa a sus hijos internos.[17] Este patrón imita elementos nativos de HTML como <select> y <option>, donde el estado se comparte implícitamente.

4.1.1 Solucionando APIs Rígidas

En un enfoque tradicional, un componente Modal podría aceptar props como title, footerButtons, onClose, headerStyle, bodyClassName, showCloseButton, etc. Esto conduce a una rigidez extrema; si un desarrollador necesita agregar un subtítulo o cambiar el orden de los botones, debe modificar la implementación interna del componente Modal.

El patrón de Componentes Compuestos invierte este control. El componente padre (Modal)

gestiona el estado (abierto/cerrado), mientras que los subcomponentes (Modal.Header, Modal.Body, Modal.Footer) manejan el renderizado. El padre utiliza React.Context para proveer el estado y los manejadores a estos hijos de manera implícita, sin que el consumidor de la API necesite pasar props manualmente.[19]

4.1.2 Implementación y Beneficios de Código Limpio

Este patrón mejora drásticamente la legibilidad y la flexibilidad. El consumidor compone la UI de manera declarativa. También refuerza el principio **Open/Closed** de SOLID: el componente está abierto a la extensión (componiendo los hijos de formas nuevas) pero cerrado a la modificación (la lógica interna no cambia).[21]

Ejemplo Arquitectónico:

TypeScript

```
// Consumo Limpio
<Tabs>
  <Tabs.List>
    <Tabs.Trigger value="account">Cuenta</Tabs.Trigger>
    <Tabs.Trigger value="password">Contraseña</Tabs.Trigger>
  </Tabs.List>
  <Tabs.Content value="account">><AccountForm />></Tabs.Content>
  <Tabs.Content value="password">><PasswordForm />></Tabs.Content>
</Tabs>
```

Este código es auto-documentado y resistente a cambios en el diseño, a diferencia de una configuración basada en objetos JSON pasados como props.[22]

4.2 Refactorización mediante Hooks Personalizados (Custom Hooks)

La refactorización de componentes grandes a menudo se aborda incorrectamente dividiendo el método render en funciones auxiliares (renderHeader(), renderBody()). Esto es un

anti-patrón que oscurece las dependencias, causa problemas de rendimiento (las funciones se recrean en cada render) y mantiene la lógica acoplada a la vista. La estrategia superior es la extracción de la "Lógica de Negocio" hacia Hooks Personalizados.[23]

4.2.1 Separación de Lógica y Vista

Un componente idealmente debe funcionar como una capa de vista pura. Todos los efectos secundarios (useEffect), llamadas a API y manipulaciones de estado complejas deben encapsularse en hooks. Para un componente UserDashboard, la lógica como obtener datos de usuarios, filtrar activos y manejar errores debe residir en useUserDashboard o hooks granulares como useUsers y useActiveFilter.

Esta separación permite:

1. **Testabilidad:** Los hooks pueden probarse en aislamiento usando librerías como react-hooks-testing-library sin necesidad de renderizar el DOM completo.
2. **Reutilización:** La lógica de obtención de datos puede reutilizarse en otras vistas.
3. **Legibilidad:** El cuerpo del componente se convierte en una descripción declarativa de la UI, libre de la carga cognitiva de múltiples useEffect y lógica imperativa.[25]

4.2.2 La Capa de Abstracción del Hook

Al refactorizar, se deben buscar clústeres de useState y useEffect que cambien juntos. Estos son candidatos para extracción. Por ejemplo, una lógica de manejo de formularios con validación es un candidato principal para un hook useFormValidator. Esto reduce la carga cognitiva al leer el archivo del componente, ya que el "cómo" (detalles de implementación) se oculta detrás del "qué" (el nombre semántico del hook).[27]

---

5. La Capa de Servicio API: Estructurando el Acceso a Datos

Un fallo crítico en muchas aplicaciones React es la dispersión de llamadas fetch o axios directamente dentro de los componentes. Esto conduce a duplicación de código, manejo

inconsistente de errores y dificultades extremas para realizar pruebas unitarias. Una arquitectura robusta exige una Capa de Servicio API centralizada.

5.1 Enfoque de Arquitectura en Capas

La arquitectura debe seguir una jerarquía estricta para desacoplar la UI de los datos:

1. **Capa de Entorno:** Gestión centralizada de URLs base y variables de entorno.
2. **Capa de Cliente HTTP:** Una instancia configurada (ej. Axios) con interceptores para tokens de autenticación, logging y manejo global de errores.
3. **Capa de Servicio:** Funciones puras que definen los endpoints y los tipos de retorno/entrada.
4. **Capa de Hooks:** Integración de React Query (TanStack Query) que consume la capa de servicio.
5. **Capa de Componente:** UI pura que consume los hooks personalizados.

Esta separación asegura que los componentes nunca se comuniquen directamente con la red, sino con una abstracción de la misma.[28]

5.2 Cliente API Centralizado e Interceptores

Crear una instancia única de Axios permite centralizar preocupaciones transversales. En lugar de agregar manualmente Authorization: Bearer token a cada solicitud, un interceptor lo inyecta automáticamente. De manera similar, el manejo global de errores (como redirigir al login en un estado 401) debe manejarse aquí.

Mejor Práctica: No exportar la instancia cruda de axios a los componentes. Solo la Capa de Servicio debe importar el cliente. Esto previene que los desarrolladores eviten los métodos de conveniencia y las validaciones de la capa de servicio.[28]

5.3 Integración de React Query (TanStack Query)

Mientras que la Capa de Servicio maneja la *promesa* de los datos, React Query maneja el *estado* de esos datos (cargando, error, caché, revalidación). En 2025, usar useEffect para la

obtención de datos se considera un anti-patrón para interacciones REST estándar debido a la complejidad de manejar condiciones de carrera (race conditions) y caché manualmente.[31]

5.3.1 El Patrón de Hook de Consulta Personalizado

Al igual que las llamadas fetch no deben estar en los componentes, las configuraciones de useQuery no deben estar dispersas. Se debe encapsular el uso de useQuery en hooks personalizados dentro de la capa API.

Patrón de Implementación:

TypeScript

```
// api/users.ts (Capa de Servicio)
export const getUser = (id: string) => apiClient.get<User>(`/users/${id}`).then(res => res.data);

// hooks/useUser.ts (Capa de Hooks)
export const useUser = (id: string) => {
  return useQuery({
    queryKey: ['user', id], // Key consistente y centralizada
    queryFn: () => getUser(id),
    staleTime: 5 * 60 * 1000, // Configuración específica de la entidad
  });
};
```

Esto asegura que la Query Key (['user', id]) sea consistente en toda la aplicación, evitando desajustes en la caché. También permite centralizar configuraciones como staleTime o lógica de reintento específica para esa entidad de datos.[33]

5.4 Wrappers Genéricos de Respuesta API

Para mantener el código limpio en TypeScript y evitar la repetición, es efectivo definir un wrapper genérico para las respuestas de la API. La mayoría de las APIs devuelven un sobre estándar (ej. { data: T, status: number, message: string }). Definir esto una vez previene el

tipado repetitivo y propenso a errores.

TypeScript

```typescript
interface ApiResponse<T> {
  data: T;
  status: number;
  message: string;
  timestamp: string;
}

// Función genérica de solicitud
async function request<T>(url: string): Promise<T> {
  const response = await fetch(url);
  // Inferencia automática del tipo de respuesta completa
  const result: ApiResponse<T> = await response.json();
  return result.data; // Desempaquetado automático
}
```

Este enfoque genérico permite que la inferencia de tipos fluya desde la definición de la API a través de la capa de servicio hasta el componente de UI, asegurando una seguridad de tipos de extremo a extremo sin necesidad de castings manuales intermedios.[35]

---

6. Organización Estructural y Code Splitting

La estructura de carpetas de un proyecto React impacta significativamente su escalabilidad y la carga cognitiva del desarrollador. El debate típico se sitúa entre "agrupar por tipo" (carpetas de componentes, hooks, estilos separadas) y "agrupar por característica".

6.1 Arquitectura Basada en Características (Feature-Based)

Para aplicaciones escalables, la estructura "Basada en Características" (Feature-Based) o guiada por el dominio es superior. Esto implica agrupar todos los archivos relacionados con

una característica de negocio específica (componentes, hooks, tipos, tests, estilos) en un solo directorio.

Estructura Recomendada:

```
src/
  features/
    Auth/
      components/
        LoginForm.tsx
        RegisterForm.tsx
      hooks/
        useAuth.ts
      api/
        authService.ts
      types.ts
      index.ts (API pública del feature)
    Dashboard/
  components/ (UI genérica compartida: Botones, Modales base)
  hooks/ (Hooks genéricos compartidos: useDebounce, useClickOutside)
```

Esta estructura se adhiere al principio de **Colocación (Colocation)**: las cosas que cambian juntas deben permanecer juntas. Minimiza el cambio de contexto al navegar por el código y hace que eliminar una característica obsoleta sea tan simple como borrar una carpeta, previniendo el "código huérfano" que suele acumularse en carpetas generales de components.[31]

6.2 El Patrón de Archivo Barril (Barrel File index.ts)

El uso de archivos index.ts para exportar la API pública de una carpeta de características es una técnica poderosa de encapsulamiento. Permite que el feature exponga solo lo necesario (por ejemplo, el componente contenedor principal) mientras mantiene privados los subcomponentes y ayudantes internos.

Sin embargo, se aconseja precaución: el uso imprudente de archivos barril puede causar problemas de "dependencias circulares" y aumentar el tamaño del bundle si el "tree-shaking"

(eliminación de código muerto) no está configurado correctamente en el empaquetador. Es una buena práctica usarlos en los límites del feature pero evitarlos para anidamientos internos profundos.[38]

6.3 Code Splitting y Carga Perezosa (Lazy Loading)

El code splitting debe implementarse al nivel de las rutas principales. Usar React.lazy permite que la aplicación cargue solo el JavaScript requerido para la vista actual, reduciendo drásticamente el tiempo de carga inicial.

Mejor Práctica:
Envolver los componentes de ruta en un componente genérico LazyLoader o usar un layout que maneje el Suspense fallback centralizadamente. Esto mantiene limpia la configuración del router.

TypeScript

```
// Router.tsx
const Dashboard = React.lazy(() => import('@/features/Dashboard'));

const App = () => (
  <Suspense fallback={<GlobalLoadingSpinner />}>
    <Routes>
      <Route path="/dashboard" element={<Dashboard />} />
    </Routes>
  </Suspense>
);
```

Combinar esto con la estructura de carpetas basada en características facilita identificar qué "chunks" (fragmentos de código) corresponden a qué lógica de negocio en el análisis del bundle.[31]

---

7. Semántica del Código Limpio: Nombrado y Convenciones

El nombrado es el indicador más abstracto pero inmediato de la calidad del código. En React y TypeScript, las convenciones de nombrado consistentes reducen la carga cognitiva y sirven como documentación implícita.

7.1 Variables Booleanas

Las variables booleanas siempre deben responder a una pregunta de sí/no. Los prefijos como is, has, y should son obligatorios para distinguir valores de funciones o strings.

- isOpen (Estado: ¿Está abierto?)
- hasError (Estado derivado: ¿Tiene error?)
- shouldRender (Lógica: ¿Debería renderizarse?)

Nombres ambiguos como open, error, o render deben evitarse estrictamente, ya que obligan al lector a inspeccionar el tipo para entender si es una acción, un estado o un objeto.[39]

7.2 Manejadores de Eventos: on vs. handle

Se debe hacer una distinción estricta entre la *prop* pasada a un componente y la *función* que la implementa.

- **Props:** Usar el prefijo on (ej. onClick, onSubmit). Esto imita la API nativa del DOM e indica una suscripción a un evento.
- **Manejadores (Handlers):** Usar el prefijo handle (ej. handleClick, handleSubmit). Esto indica la implementación de la lógica de respuesta.

Ejemplo de Semántica Correcta:

TypeScript

```
// Contrato (Interface)
interface ButtonProps {
  onClick: () => void; // Es un evento
```

```
}

// Implementación
const SubmitButton = ({ onClick }: ButtonProps) => {
  // Implementación interna
  const handleClick = () => {
    trackAnalytics('button_click');
    onClick(); // Invoca la prop
  };
  return <button onClick={handleClick}>Enviar</button>;
}
```

Esta convención desambigua la fuente de la función (externa vía prop vs. interna) y se alinea con los estándares de la comunidad de React.[39]

7.3 Genéricos Descriptivos

En los genéricos de TypeScript, variables de una sola letra como T son aceptables para tipos utilitarios simples y abstractos. Sin embargo, en lógica de dominio compleja, se prefieren nombres genéricos descriptivos. TData, TResponse, TProps proporcionan contexto que T carece, especialmente cuando hay múltiples genéricos involucrados (<TData, TError>). Esto mejora la legibilidad de las firmas de función complejas y ayuda a los nuevos desarrolladores a entender qué representa el tipo genérico.[35]

---

8. Técnicas Avanzadas de Tipado y Refactorización

8.1 Uniones Discriminadas para Gestión de Estado

Una de las características más poderosas de TypeScript para desarrolladores React es la Unión Discriminada. Este patrón es particularmente efectivo para gestionar estados de UI complejos (Cargando, Éxito, Error) dentro de un reducer o estado, eliminando combinaciones de estado inválidas.

En lugar de tener booleanos separados (isLoading, isError, data), que teóricamente permitirían isLoading: true y isError: true simultáneamente (un estado imposible), las Uniones Discriminadas fuerzan la exclusión mutua.

Patrón de "Estados Imposibles":

TypeScript

```typescript
type State =

| { status: 'idle' }
| { status: 'loading' }
| { status: 'success'; data: User } // 'data' solo existe aquí
| { status: 'error'; error: Error }; // 'error' solo existe aquí

function reducer(state: State, action: Action): State {
  if (state.status === 'success') {
    // TypeScript sabe que 'state.data' es seguro de acceder aquí
    console.log(state.data.name);
  }
  // Acceder a state.data aquí causaría un error de compilación
}
```

Este patrón aprovecha el análisis de flujo de control de TypeScript. Dentro de un bloque donde state.status === 'success', TypeScript estrecha (narrows) el tipo automáticamente. Esto se alinea con la filosofía de "Hacer que los estados imposibles sean imposibles de representar", piedra angular del desarrollo robusto.[3]

8.2 Tipos Opacos y "Branded Types"

Para aplicaciones que manejan datos críticos (por ejemplo, distinguir entre un UserId string y un ProductId string para evitar pasarlos a la función incorrecta), los "Branded Types" (o Tipos Opacos) previenen el intercambio accidental de valores primitivos.

TypeScript

```
type UserId = string & { readonly __brand: unique symbol };
type ProductId = string & { readonly __brand: unique symbol };

const userId = "user_123" as UserId;
const productId = "prod_456" as ProductId;

function getUser(id: UserId) { /*...*/ }

// getUser(productId); // Error: Type 'ProductId' is not assignable to type 'UserId'.
```

Aunque esto añade verbosidad, proporciona una capa de seguridad para el diseño guiado por el dominio (DDD) dentro de React, asegurando que los argumentos pasados a los servicios API sean semánticamente correctos, no solo estructuralmente correctos.[1]

8.3 Estrategias de Refactorización de Legado

Refactorizar bases de código React heredadas requiere un enfoque estratégico para evitar regresiones.

1. **Patrón Strangler Fig:** Al refactorizar un componente masivo, no lo reescriba desde cero. En su lugar, cree nuevos componentes pequeños para nuevas características o partes aisladas del componente antiguo. Reemplace gradualmente el uso del componente antiguo con los nuevos hasta que el antiguo pueda eliminarse.
2. **Estrechamiento de Tipos en Fronteras:** En archivos JS heredados que se están convirtiendo a TS, evite usar any. Si el tipo es desconocido, use unknown. Esto obliga al desarrollador a realizar comprobaciones en tiempo de ejecución (type narrowing) antes de usar los datos, lo cual es más seguro y revela bugs ocultos durante el proceso de conversión.
3. **Snapshot Testing:** Antes de refactorizar un componente de UI complejo, genere un Snapshot de Jest. Esto proporciona una red de seguridad, asegurando que la refactorización (por ejemplo, extraer un hook) no altere accidentalmente la salida del DOM renderizado.[24]

---

9. Conclusión

La transición de una aplicación React funcional a un sistema limpio y escalable requiere una disciplina férrea y la adopción de patrones arquitectónicos probados. Al imponer una estricta seguridad de tipos mediante el uso adecuado de interfaces y ReactNode, construimos una base resiliente. Al adoptar una Arquitectura de API en Capas y utilizar React Query, desacoplamos nuestra UI de las complejidades del estado de la red. Finalmente, al aprovechar patrones como Componentes Compuestos y mantener rigurosas convenciones de nombrado, aseguramos que la base de código permanezca legible y maleable.

El rol del "Mentor de Código Limpio" en este contexto es hacer cumplir estas restricciones no como obstáculos burocráticos, sino como barandillas que guían al equipo de desarrollo hacia un sistema que es robusto, predecible y fácil de mantener a largo plazo.

Lista de Verificación (Checklist) para el Arquitecto React

1. **Preferir type para estado/props internos** para aprovechar uniones; **usar interface para APIs públicas**.
2. **Usar ReactNode** para props children; evitar React.FC.
3. **Centralizar llamadas API** en una Capa de Servicio dedicada; nunca hacer fetch en componentes.
4. **Usar React Query** (TanStack Query) para gestionar estado del servidor y caché.
5. **Extraer lógica a Custom Hooks** para mantener los componentes enfocados en la Vista.
6. **Usar Carpetas Basadas en Características** para colocar lógica y estilos relacionados juntos.
7. **Forzar Convenciones de Nombrado**: is/has para booleanos, handle/on para eventos.
8. **Implementar Uniones Discriminadas** para eliminar estados de UI imposibles.

Obras citadas

1. TypeScript Best Practices in 2025 - DEV Community, fecha de acceso: noviembre 29, 2025, https://dev.to/mitu_mariam/typescript-best-practices-in-2025-57hb
2. ryanmcdermott/clean-code-javascript: Clean Code ... - GitHub, fecha de acceso: noviembre 29, 2025, https://github.com/ryanmcdermott/clean-code-javascript
3. Typing Component Props | React TypeScript Cheatsheets, fecha de acceso: noviembre 29, 2025, https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/basic_type_example
4. Types vs. interfaces in TypeScript - LogRocket Blog, fecha de acceso: noviembre 29, 2025, https://blog.logrocket.com/types-vs-interfaces-typescript/
5. When to Use type vs interface in TypeScript? | by Joodi - Medium, fecha de

acceso: noviembre 29, 2025, https://joodi.medium.com/when-to-use-type-vs-interface-in-typescript-5e8c4996d140
6. Interface vs Type. What should I use in react/typescript : r/reactjs - Reddit, fecha de acceso: noviembre 29, 2025, https://www.reddit.com/r/reactjs/comments/14q6nyr/interface_vs_type_what_should_i_use_in/
7. ReactNode vs PropsWithChildren. Understanding the children Prop in... | by Tech Beast, fecha de acceso: noviembre 29, 2025, https://medium.com/@raindefiance/reactnode-vs-propswithchildren-d3f51ae35324
8. Typing Children and When to Use ReactNode | React with TypeScript | Steve Kinney, fecha de acceso: noviembre 29, 2025, https://stevekinney.com/courses/react-typescript/jsx-types-reactnode-reactelement
9. Types for React components with children - DEV Community, fecha de acceso: noviembre 29, 2025, https://dev.to/zeedu_dev/types-for-react-components-with-children-5d7k
10. Using the React children prop with TypeScript - LogRocket Blog, fecha de acceso: noviembre 29, 2025, https://blog.logrocket.com/react-children-prop-typescript/
11. React.PropsWithChildren with no props other than `children`? - Stack Overflow, fecha de acceso: noviembre 29, 2025, https://stackoverflow.com/questions/65548388/react-propswithchildren-with-no-props-other-than-children
12. Using TypeScript - React, fecha de acceso: noviembre 29, 2025, https://react.dev/learn/typescript
13. Forms and Events - React TypeScript Cheatsheets, fecha de acceso: noviembre 29, 2025, https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/forms_and_events/
14. How to type a React form onSubmit handler | Epic React by Kent C. Dodds, fecha de acceso: noviembre 29, 2025, https://www.epicreact.dev/how-to-type-a-react-form-on-submit-handler
15. Typescript: React event types - javascript - Stack Overflow, fecha de acceso: noviembre 29, 2025, https://stackoverflow.com/questions/42081549/typescript-react-event-types
16. Typescript type for form submit event - Stack Overflow, fecha de acceso: noviembre 29, 2025, https://stackoverflow.com/questions/70646761/typescript-type-for-form-submit-event
17. Mastering Compound Components Design Pattern in React || Day 03 - YouTube, fecha de acceso: noviembre 29, 2025, https://www.youtube.com/watch?v=LglWulOqh6k
18. Advanced React Patterns: Learn About Compound Components - Reddit, fecha de acceso: noviembre 29, 2025,

https://www.reddit.com/r/react/comments/1eve00s/advanced_react_patterns_learn_about_compound/
19. React Hooks: Compound Components - Kent C. Dodds, fecha de acceso: noviembre 29, 2025, https://kentcdodds.com/blog/compound-components-with-react-hooks
20. Beyond the Basics: Exploring React's Compound Components - DEV Community, fecha de acceso: noviembre 29, 2025, https://dev.to/melvinprince/beyond-the-basics-exploring-reacts-compound-components-1m0
21. React Design Patterns: Compound Component Pattern | by Vitor Britto - Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/@vitorbritto/react-design-patterns-compound-component-pattern-ec247f491294
22. How to Use the Compound Components Pattern in React: Prop Soup to Flexible UIs, fecha de acceso: noviembre 29, 2025, https://www.freecodecamp.org/news/compound-components-pattern-in-react/
23. Refactoring React: Taming Chaos, One Component at a Time - DEV Community, fecha de acceso: noviembre 29, 2025, https://dev.to/vigneshiyergithub/refactoring-react-taming-chaos-one-component-at-a-time-df
24. Common Sense Refactoring of a Messy React Component - Alex Kondov, fecha de acceso: noviembre 29, 2025, https://alexkondov.com/refactoring-a-messy-react-component/
25. Refactoring components in React with custom hooks - CodeScene, fecha de acceso: noviembre 29, 2025, https://codescene.com/blog/refactoring-components-in-react-with-custom-hooks
26. Enhance Large React Components with Custom Hooks for Business Logic - Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/recur-club-engineering/enhance-large-react-components-with-custom-hooks-for-business-logic-fc5305f86788
27. Why react hooks are better than classes? : r/reactjs - Reddit, fecha de acceso: noviembre 29, 2025, https://www.reddit.com/r/reactjs/comments/1cdtvxu/why_react_hooks_are_better_than_classes/
28. REST API Architecture in React: A Production Guide | by Kennedy Owusu | Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/@kennediowusu/rest-api-architecture-in-react-a-production-guide-966f663522ba
29. The Most Effective Method For Managing API Calls In Your React Or React Native Project, fecha de acceso: noviembre 29, 2025, https://dhruvpvx.medium.com/the-most-effective-method-for-managing-api-calls-in-your-react-or-react-native-project-fe4293a7905f
30. Where is the most optimal placing of fetch requests in React? : r/reactjs - Reddit, fecha de acceso: noviembre 29, 2025,

https://www.reddit.com/r/reactjs/comments/1jbupgh/where_is_the_most_optimal_placing_of_fetch/
31. What do you think is the best practices when working with React projects? - Stack Overflow, fecha de acceso: noviembre 29, 2025, https://stackoverflow.com/questions/79830882/what-do-you-think-is-the-best-practices-when-working-with-react-projects
32. fgnass/react-api-query: Hooks to use react-query with a typed API client - GitHub, fecha de acceso: noviembre 29, 2025, https://github.com/fgnass/react-api-query
33. React-Query Best Practices: Separating Concerns with Custom Hooks | by Majid Lotfinia, fecha de acceso: noviembre 29, 2025, https://majidlotfinia.medium.com/react-query-best-practices-separating-concerns-with-custom-hooks-3f1bc9051fa2
34. Tanstack react query architecture : r/reactjs - Reddit, fecha de acceso: noviembre 29, 2025, https://www.reddit.com/r/reactjs/comments/18z3nsi/tanstack_react_query_architecture/
35. How to wrap your head around Typescript generics - freeCodeCamp, fecha de acceso: noviembre 29, 2025, https://www.freecodecamp.org/news/how-to-wrap-your-head-around-typescript-generics-8d243f7de78/
36. Building a Typesafe API Wrapper - Daniel Craciun, fecha de acceso: noviembre 29, 2025, https://www.danielfullstack.com/article/building-a-typesafe-api-wrapper
37. Structured API Calls in React. In modern web development, building... | by Sankalpa Neupane | Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/@sankalpa115/structured-api-calls-in-react-ef6fa8f8681b
38. How To Structure React Projects From Beginner To Advanced, fecha de acceso: noviembre 29, 2025, https://blog.webdevsimplified.com/2022-07/react-folder-structure/
39. Naming Conventions in React for Clean & Scalable Code - Sufle, fecha de acceso: noviembre 29, 2025, https://www.sufle.io/blog/naming-conventions-in-react
40. React JS - Naming convention - DEV Community, fecha de acceso: noviembre 29, 2025, https://dev.to/kristiyanvelkov/react-js-naming-convention-lcg
41. Naming Conventions in React for Clean & Scalable Code | by Sufle.io - Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/@sufleio/naming-conventions-in-react-for-clean-scalable-code-f6de31294452
42. What is the right name of event handler? onClick or handleClick? [closed] - Stack Overflow, fecha de acceso: noviembre 29, 2025, https://stackoverflow.com/questions/60048249/what-is-the-right-name-of-event-handler-onclick-or-handleclick
43. How I refactored a 2700-line React component at my self-driving car company - Reddit, fecha de acceso: noviembre 29, 2025, https://www.reddit.com/r/reactjs/comments/vswcvf/how_i_refactored_a_2700line_react_component_at_my/